

QuantumLearn AI

Complete Project and Architecture Report

A source-based explanation of the application, file structure, technology stack, runtime flow, data model, user journeys, and current engineering risks.

Repository: leelaprasad55/quantumai

Branch snapshot: main

Latest pushed commit: 74317bc

Report date: September 4, 2026

Executive summary: QuantumLearn AI is a React/Vite single-page quantum-computing learning platform. React renders the interface, Supabase manages identity and persistent data, a custom JavaScript engine simulates small quantum circuits locally, and a Supabase Edge Function connects the AI tutor to Groq.

1. Project at a Glance

Layer	Implementation
Frontend	React 19 functional components and hooks
Build	Vite 8 with the React plugin
Navigation	React Router DOM 7
Styling	Vanilla CSS with CSS variables and responsive layouts
Authentication	Supabase Auth with email/password and Google OAuth
Persistence	Supabase PostgreSQL, Storage, RPC functions, plus localStorage fallback
AI	Groq through a Supabase Edge Function, direct fallback, and offline responses
Quantum logic	Custom browser-side state-vector simulator
Visualizations	Canvas Bloch sphere and Chart.js charts

2. File-by-File Map

Path	Responsibility
index.html	HTML shell, metadata, fonts, KaTeX stylesheet, React root.
package.json	Dependencies and scripts: dev, build, preview.
vite.config.js	Vite and React plugin configuration.
src/main.jsx	React entry point, StrictMode, BrowserRouter, global CSS.
src/App.jsx	Providers, routes, auth guards, loading screen, sidebar shell.
src/styles/index.css	Global dark glassmorphic theme, layout, forms, animations, circuit UI.
src/context/AuthContext.jsx	Registration, login, OAuth, session restoration, profile updates, logout.
src/context/ProgressContext.jsx	Loads progress/skills and exposes completion, scoring, and analytics actions.
src/lib/supabaseClient.js	Creates the Supabase browser client from environment variables.
src/utils/storage.js	Supabase data-access wrapper for progress, circuits, labs, chat, avatars, and admin data.
src/utils/quantum.js	Complex arithmetic, gates, state vectors, measurement, Bloch math, circuit explanation.
src/utils/adaptive.js	Knowledge scoring, skill updates, prerequisite checks, personalized roadmap generation.
src/utils/aiTutor.js	Edge Function call, direct Groq fallback, circuit explanation, offline tutor responses.
src/components/layout/Sidebar.jsx	Authenticated navigation, score display, avatar upload, logout.
src/components/ai/AITutor.jsx	Floating tutor UI, chat history, context, response rendering.
src/components/BlochSphere.jsx	Canvas 2D animated Bloch sphere with mouse rotation.
src/components/CircuitExplainer.jsx	Four-tab circuit analysis and AI explanation drawer.

<code>src/components/CircuitPuzzle.jsx</code>	Reusable target-state circuit puzzle component; currently not connected to the main flow.
<code>src/pages/Auth/AuthPages.jsx</code>	Login, registration, education, goal, and OAuth forms.
<code>src/pages/KnowledgeTest/KnowledgeTest.jsx</code>	Initial skill assessment and onboarding completion.
<code>src/pages/Dashboard/Dashboard.jsx</code>	Home dashboard with progress, next module, skills, and quick actions.
<code>src/pages/Roadmap/Roadmap.jsx</code>	Personalized module path visualization.
<code>src/pages/Module/ModuleList.jsx</code>	24-module catalogue and completion/prerequisite presentation.
<code>src/pages/Module/ModuleDetail.jsx</code>	Topics, learning links, formulas, quizzes, challenges, and module assessment.
<code>src/pages/CircuitBuilder/CircuitBuilder.jsx</code>	Interactive circuit editor, simulator, save/load, share, measurement, explain.
<code>src/pages/QuantumLab/QuantumLab.jsx</code>	Preset/custom experiments, noise, histograms, code generation, saved labs.
<code>src/pages/QuantumRace/QuantumRace.jsx</code>	Classical versus quantum algorithm animation and comparison.
<code>src/pages/SkillMap/SkillMap.jsx</code>	14-skill radar visualization.
<code>src/pages/Achievements/Achievements.jsx</code>	Badge gallery based on progress conditions.
<code>src/pages/Admin/AdminPanel.jsx</code>	Users, cohorts, modules, analytics, activity, gate usage, reset controls.

3. Static Data Files

- `src/data/modules.js`: 24 modules, categories, prerequisites, goals, education levels, skills, achievements.
- `src/data/topics.js`, `topics2.js`, `topics3.js`, `topics4.js`: topic records for modules 1-24.
- `src/data/allTopics.js`: combined topic index.
- `src/data/questions.js`: initial knowledge-test question bank.
- `src/data/topicQuestions.js`: module/topic quiz questions and fallback question generation.
- `src/data/topicCircuits.js`: topic circuit challenge definitions.
- `src/data/topicFormulas.js`: formula lookup content.

4. Runtime Architecture

Browser loads index.html

- > src/main.jsx mounts React
- > BrowserRouter enables routes
- > AuthProvider restores Supabase session
- > ProgressProvider loads progress and skills
- > App selects public or authenticated layout
- > Page components call contexts, utilities, and Supabase
- > React state updates the interface
- > Supabase persists user data

5. Authentication Flow

1. Registration calls `supabase.auth.signUp()` with name, education, goal, and avatar metadata.
2. A database trigger creates a profile with the safe default role `student`.
3. Login calls `signInWithPassword()`; Google login uses Supabase OAuth.
4. `AuthContext` fetches the matching profile and creates the frontend user object.
5. New users are redirected to `/test`; assessed users go to `/dashboard`.
6. Logout ends the Supabase session and clears local user state.

6. Learning and Progress Flow

Register/login

- > knowledge assessment
- > skill percentages calculated
- > personalized roadmap generated
- > module opened
- > topic learned and marked complete
- > topic_progress saved
- > module assessment submitted
- > module_progress saved
- > activity and skill data updated
- > dashboard, roadmap, skill map, achievements refresh

7. Adaptive Learning

`generateRoadmap()` averages the 14 skills. Beginners below 20 receive the full 24-module path. More advanced learners receive modules related to weak skills and their selected goal, with completed modules removed and prerequisites considered. Module 24 is always appended.

8. Quantum Circuit Flow

User places gates

- > operation list changes
- > `simulateCircuit()` initializes $|000\dots0\rangle$
- > operations are applied as matrix-vector multiplications
- > final amplitudes become probabilities
- > charts, Bloch sphere, state text, and measurements update
- > optional save, share, gate analytics, or AI explanation

The simulator is local JavaScript. It supports H, X, Y, Z, S, T, Rx, Ry, Rz, CNOT, SWAP, CZ, measurement, tensor products, reduced states, and global-phase-aware comparisons.

9. AI Tutor Flow

Student asks question

- > Supabase ai-tutor Edge Function
- > server-side GROQ_API_KEY
- > Groq model fallback chain
- > Markdown response
- > chat message persisted
- > if unavailable: direct Groq key, then offline keyword response

The circuit explainer uses the same server path and can fall back to a local structured explanation generated from the simulator.

10. Quantum Lab and Race

The Quantum Lab supports educational presets such as Bell, GHZ, W, teleportation, Grover, QPE, and Deutsch-Jozsa, as well as noise models, histograms, fidelity, and code generation for Qiskit, PennyLane, Cirq, and OpenQASM.

The hardware dispatch screen is a local simulation. It creates a fake job ID and uses timers; it does not submit to IBM Quantum, Azure Quantum, Amazon Braket, or another real provider.

The Quantum Race animates classical and quantum complexity comparisons for Grover, Shor, Deutsch-Jozsa, Simon, HHL, and QFT.

11. Database Model

Table	Purpose
profiles	Identity profile, role, XP, streak, knowledge score.
user_progress	General counters, current module/topic, time, experiments.
user_skills	Fourteen skill percentages.
module_progress, topic_progress	Completion records and scores.
test_attempts	Assessment history.
saved_circuits, lab_experiments	Saved interactive work.
activity_log, gate_usage	User and global analytics.
puzzle_progress	Circuit challenge completion.
ai_chat_messages	AI conversation history.

supabase/migrations/01_init_schema.sql creates the schema. 02_rls_policies.sql enables Row Level Security. 03_admin_rpcs.sql contains protected admin functions. 04_user_rpcs.sql contains narrow user functions. 05_add_constraints.sql adds data constraints. 06_avatar_storage.sql configures avatar storage.

12. Security and Engineering Risks

Credential warning: A live Groq key was previously found in the environment template during project preparation. It was removed from the tracked template, but the key must be revoked and replaced. Any `VITE_*` value is visible in the browser bundle, so Groq secrets must live only in Supabase Edge Function secrets.

- AI responses are rendered with `dangerouslySetInnerHTML` without a proper Markdown sanitizer.
- The AI Edge Function uses wildcard CORS and has no explicit caller authorization or rate limiting.
- User-entered API keys are stored in `localStorage`.
- Lab fidelity is displayed as 0-100 but the database constraint expects 0-1.
- Lab experiment counters, XP, streaks, and some question counters are not consistently incremented.
- Some quizzes display scores without persisting them through `recordTestScore()`.
- Prerequisites are shown in the UI but are not fully enforced for direct module URLs.
- The reusable circuit puzzle is not connected to the main application flow.
- The topic circuit challenge has operation/amplitude handling inconsistencies.
- Some admin cohort and gradebook data is generated in the frontend and is not durable.
- There are no automated tests, lint checks, or type checks.

13. Documentation Mismatch

The current `README.md` describes an older version of the application. It says persistence is localStorage-only and describes real hardware, WebGL, older authentication, and other behavior that does not match the current source. The source implementation and Supabase migrations should be treated as authoritative until the README is rewritten.

14. Validation Status

The production build completed successfully with `npm run build`. Vite reported only deprecation and bundle-size warnings. No automated runtime or integration tests are currently configured.

15. Recommended Next Steps

1. Revoke and rotate the exposed Groq credential.
2. Remove direct browser Groq access and use only the secured Edge Function.
3. Add Markdown parsing with HTML sanitization.
4. Fix fidelity unit conversion and lab experiment counting.
5. Repair assessment, XP, streak, puzzle, and question-counter persistence.
6. Enforce prerequisites in route/module access.
7. Integrate and test the circuit puzzle flow.
8. Add unit and integration tests for authentication, progress, simulator math, and AI fallback behavior.
9. Rewrite the README to match the Supabase-backed implementation.

This report was generated from the current workspace source, configuration, migrations, and build status. It describes implemented behavior and explicitly labels simulated or incomplete features.